Claude Chat

Share link: https://claude.ai/share/a8bd394e-2235-4069-a093-f1d3b91b7927

Shared copy of a chat between Claude and Mohamed. Content may include unverified or unsafe content that does not represent the views of Anthropic. The chat text and all code blocks below are shown as in the shared copy.

PROMPT 1

Hi! I'm a computer science student and I'm working on an OOP assignment in C++. I need to design a simulator for a made-up computer called the Vole machine (it comes from Brookshear's "Computer Science: An Overview" book). For now I only need the DESIGN, not the code. I want a UML class diagram written in PlantUML.

Let me explain the machine first.

The Vole machine has:

- A memory of 256 cells. Each cell holds 1 byte, and the addresses go from 00 to FF in hex. The cell at address FF is special: it is connected to the screen, so writing to FF means printing to the screen.
- 16 general-purpose registers, numbered 0 to F, each holding 1 byte.
- A Program Counter (PC), which holds the address of the next instruction, and an Instruction Register (IR), which holds the current instruction.
- Every instruction is 2 bytes (4 hex digits), so it takes two memory cells.
- The CPU repeats a cycle: fetch the instruction at PC into IR (and move PC forward by 2), decode it, then execute it.

The instructions I need to support are (R, S, T are register numbers, and XY is a hex value or address):

1 RXY: load register R from memory cell XY
2 RXY: load register R with the value XY
3 RXY: store register R into memory cell XY
3 RFF: store register R to cell FF, which means print to the screen
4 0RS: copy register R into register S
5 RST: add registers S and T as two's complement integers, put the result in R
6 RST: add registers S and T as floating-point numbers, put the result in R
7 RST: R = S OR T
8 RST: R = S AND T
9 RST: R = S XOR T
A R0X: rotate register R to the right X times (circular rotation)
B RXY: if register R equals register 0, jump to address XY (set PC = XY), otherwise just continue
C 000: halt

Here is what the simulator should do:

1. Show a menu of choices.
2. Load a program from a text file into memory, starting at an address the user chooses. That address must be an even number (00, 02, 04, ..., F0, F2, ...) and less than FF.

3. Let the user overwrite any register or any memory cell with a new value.
4. Set the PC to the start address of the program.
5. Show the memory as a 16x16 table, plus the PC, the IR and all 16 registers.
6. Run the program by fetching each instruction into IR, checking that it is a valid instruction, decoding it, executing it and updating the PC.
7. Let the user choose between running the whole program at once or step by step.
8. If it runs all at once, show the memory, PC, IR, registers and the screen output when it finishes.
9. If it runs step by step, show all of that after every step, and wait for the user to press Enter before the next step.
10. When the program finishes, ask the user if they want to exit or load another program.
11. Invalid instructions must be rejected, and a program that contains invalid instructions must be rejected too.
The program file has one instruction per line, written as 4 hex digits (for example 2005). Lines that start with ; are comments and should be ignored, empty lines are ignored too, and anything after the 4 hex digits on a line is ignored.
I already tried to design this myself. This is what I came up with:
Classes:
1. Memory
- Responsibility: stores the 256 memory cells.
- Attributes: cells (array of 256 bytes)
- Methods: read(address), write(address, value), clear()
Register
Responsibility: stores the 16 general-purpose registers.
Attributes: regs (array of 16 bytes)
Methods: read(index), write(index, value), clear()
CPU
Responsibility: runs the fetch-decode-execute cycle.
Attributes: programCounter, instructionRegister, registers (Register), memory (Memory)
Methods: fetch(), decode(), execute(), step(), run()
ALU
Responsibility: does the arithmetic and logic operations.
Methods: addInt(a, b), addFloat(a, b), orOp(a, b), andOp(a, b), xorOp(a, b), rotate(value, times)
Machine
Responsibility: owns the CPU and the memory, and loads programs.
Attributes: cpu (CPU), memory (Memory)
Methods: loadProgram(filename, startAddress), run(), runStep(), overwriteRegister(index, value), overwriteMemory(address, value)
UI
Responsibility: the menu and all printing.
Attributes: machine (Machine)

> Methods: showMenu(), displayMemory(), displayRegisters(), displayScreen(), start()
> Relations:
> - Machine has one CPU and one Memory (composition).
> - CPU has one Register set and one ALU (composition) and uses Memory (association).
> - UI uses Machine (association).
> Could you please:
> 1. Look at my design and tell me honestly what is good and what could be better.
> 2. Give me your own best design as PlantUML code, with the classes, attributes, methods (including parameter and return types), visibility and the relationships between classes (inheritance, composition, association, dependency), with multiplicities where they make sense.
> 3. Explain in simple words what each class is responsible for and why you chose to structure it this way.
> Please try to follow good OOP principles like encapsulation and single and send it as a text not as a file responsibility. Thanks!

RESPONSE 1

1. Review of your design

What's good

- You split the machine along hardware lines (memory, registers, ALU, CPU), so each class is small and cohesive.
- The UI is separate from the logic, so the model never calls cout.
- Composition is used where the part cannot live without the whole (Machine → CPU/Memory, CPU → registers). Having the CPU only use a Memory owned by Machine is also right.

What could be better

- There is no Instruction concept. Your decode() has nothing to return, and execute() would become a big switch over 12 opcodes. That makes the CPU a "god class" and breaks the Open/Closed principle. Validation (requirements 6 and 11) also has no home.
- There is no Screen class. displayScreen() is in the UI, but the screen output has to be stored somewhere. Writing to cell FF is memory-mapped I/O, which is hardware behaviour and should live in the model, not in the UI.
- Machine does too much. Reading and validating a program file is its own responsibility. It also needs to be all-or-nothing: if any line is invalid, nothing should be written to memory.
- UI mixes input handling (menu) with output formatting (tables). I would split these into two classes.
- Register is a misleading name. It is really a register file (16 registers). PC and IR as plain public ints would break encapsulation, so keep them private with accessors. IR is 2 bytes wide, not 1.
- ALU has no state, so "CPU owns an ALU" (composition) describes a lifetime relationship that doesn't exist. Make it a utility class with static methods and use a dependency instead.
- Error handling is not designed. Invalid instructions and bad files should be reported with dedicated exception types.

- Types are vague. Use uint8_t and uint16_t so a “byte” is really a byte.

2. PlantUML design

PlantUML

```
@startuml VoleMachine
skinparam classAttributeIconSize 0
skinparam shadowing false
hide empty members

' ==================== HARDWARE ====================
package "Hardware" {

  class Screen {
    - buffer : std::string
    + print(value : uint8_t) : void
    + getContent() : const std::string&
    + clear() : void
  }

  class Memory {
    + {static} SIZE : size_t = 256
    + {static} SCREEN_ADDRESS : uint8_t = 0xFF
    - cells : std::array<uint8_t, 256>
    - screen : Screen&
    + Memory(screen : Screen&)
    + read(address : uint8_t) : uint8_t
    + write(address : uint8_t, value : uint8_t) : void
    + clear() : void
  }

  class RegisterFile {
    + {static} COUNT : size_t = 16
    - regs : std::array<uint8_t, 16>
    + read(index : uint8_t) : uint8_t
    + write(index : uint8_t, value : uint8_t) : void
    + clear() : void
  }

  class ALU <<utility>> {
    + {static} addTwosComplement(a : uint8_t, b : uint8_t) : uint8_t
    + {static} addFloat(a : uint8_t, b : uint8_t) : uint8_t
    + {static} bitOr(a : uint8_t, b : uint8_t) : uint8_t
    + {static} bitAnd(a : uint8_t, b : uint8_t) : uint8_t
    + {static} bitXor(a : uint8_t, b : uint8_t) : uint8_t
    + {static} rotateRight(value : uint8_t, times : uint8_t) : uint8_t
    - {static} floatToReal(bits : uint8_t) : double
    - {static} realToFloat(x : double) : uint8_t
  }

  interface ExecutionContext {
    + {abstract} readRegister(index : uint8_t) : uint8_t
    + {abstract} writeRegister(index : uint8_t, value : uint8_t) : void
    + {abstract} readMemory(address : uint8_t) : uint8_t
    + {abstract} writeMemory(address : uint8_t, value : uint8_t) : void
    + {abstract} setProgramCounter(address : uint8_t) : void
    + {abstract} halt() : void
  }

  class CPU {
    - pc : uint8_t
    - ir : uint16_t
    - halted : bool
    - registers : RegisterFile
    - memory : Memory&
    - current : std::unique_ptr<Instruction>
    + CPU(memory : Memory&)
```

```
    + restart(startAddress : uint8_t) : void
    + step() : void
    + run() : void
    + isHalted() : bool
    + getPC() : uint8_t
    + getIR() : uint16_t
    + getRegisters() : const RegisterFile&
    + readRegister(index : uint8_t) : uint8_t
    + writeRegister(index : uint8_t, value : uint8_t) : void
    + readMemory(address : uint8_t) : uint8_t
    + writeMemory(address : uint8_t, value : uint8_t) : void
    + setProgramCounter(address : uint8_t) : void
    + halt() : void
    - fetch() : void
    - decode() : void
    - execute() : void
  }
}

' ==================== INSTRUCTIONS ====================
package "Instructions" {

  abstract class Instruction {
    # raw : uint16_t
    # Instruction(raw : uint16_t)
    + getRaw() : uint16_t
    + toHexString() : std::string
    + {abstract} execute(ctx : ExecutionContext&) : void
  }

  abstract class BinaryRegisterInstruction {
    # r : uint8_t
    # s : uint8_t
    # t : uint8_t
    + execute(ctx : ExecutionContext&) : void
    # {abstract} compute(a : uint8_t, b : uint8_t) : uint8_t
  }

  class LoadFromMemory {
    - r : uint8_t
    - address : uint8_t
    + execute(ctx : ExecutionContext&) : void
  }
  class LoadImmediate {
    - r : uint8_t
    - value : uint8_t
    + execute(ctx : ExecutionContext&) : void
  }
  class StoreToMemory {
    - r : uint8_t
    - address : uint8_t
    + execute(ctx : ExecutionContext&) : void
  }
  class CopyRegister {
    - r : uint8_t
    - s : uint8_t
    + execute(ctx : ExecutionContext&) : void
  }
  class AddTwosComplement {
    # compute(a : uint8_t, b : uint8_t) : uint8_t
  }
  class AddFloat {
    # compute(a : uint8_t, b : uint8_t) : uint8_t
  }
  class OrRegisters {
    # compute(a : uint8_t, b : uint8_t) : uint8_t
  }
  class AndRegisters {
    # compute(a : uint8_t, b : uint8_t) : uint8_t
```

```
  }
  class XorRegisters {
    # compute(a : uint8_t, b : uint8_t) : uint8_t
  }
  class RotateRight {
    - r : uint8_t
    - times : uint8_t
    + execute(ctx : ExecutionContext&) : void
  }
  class JumpIfEqual {
    - r : uint8_t
    - address : uint8_t
    + execute(ctx : ExecutionContext&) : void
  }
  class Halt {
    + execute(ctx : ExecutionContext&) : void
  }

  class InstructionDecoder <<factory>> {
    + {static} decode(high : uint8_t, low : uint8_t) : std::unique_ptr<Instruction>
    + {static} isValid(high : uint8_t, low : uint8_t) : bool
  }

  class "std::runtime_error" as RuntimeError <<std>>
  class InvalidInstructionException {
    + InvalidInstructionException(raw : uint16_t)
  }
}

' ==================== LOADER ====================
package "Loader" {
  class ProgramLoader {
    + load(filename : const std::string&, startAddress : uint8_t, memory : Memory&) : uint8_t
    - parseFile(filename : const std::string&) : std::vector<uint16_t>
    - parseLine(line : const std::string&) : std::optional<uint16_t>
    - checkStartAddress(startAddress : uint8_t, wordCount : size_t) : void
  }
  class ProgramLoadException {
    + ProgramLoadException(message : const std::string&)
  }
}

' ==================== FACADE ====================
class Machine <<facade>> {
  - screen : Screen
  - memory : Memory
  - cpu : CPU
  - loader : ProgramLoader
  - programStart : uint8_t
  + Machine()
  + loadProgram(filename : const std::string&, startAddress : uint8_t) : void
  + overwriteRegister(index : uint8_t, value : uint8_t) : void
  + overwriteMemory(address : uint8_t, value : uint8_t) : void
  + setPC(address : uint8_t) : void
  + setPCToProgramStart() : void
  + step() : bool
  + runAll() : void
  + isFinished() : bool
  + getMemory() : const Memory&
  + getCPU() : const CPU&
  + getScreen() : const Screen&
}

' ==================== UI ====================
package "UI" {
  class ConsoleUI {
    - machine : Machine&
    - renderer : ConsoleRenderer
    + ConsoleUI(machine : Machine&)
```

```
    + start() : void
    - showMenu() : void
    - handleLoadProgram() : void
    - handleOverwriteRegister() : void
    - handleOverwriteMemory() : void
    - handleSetPC() : void
    - handleRun() : void
    - runAllAtOnce() : void
    - runStepByStep() : void
    - askExitOrLoadAnother() : bool
    - readHex(prompt : const std::string&) : uint8_t
    - readLine(prompt : const std::string&) : std::string
  }

  class ConsoleRenderer {
    + showState(machine : const Machine&) : void
    + showMemory(memory : const Memory&) : void
    + showRegisters(regs : const RegisterFile&) : void
    + showPCAndIR(pc : uint8_t, ir : uint16_t) : void
    + showScreen(screen : const Screen&) : void
    + showMessage(text : const std::string&) : void
    + showError(text : const std::string&) : void
  }
}

' ==================== RELATIONSHIPS ====================
' Ownership
Machine "1" *-- "1" Screen
Machine "1" *-- "1" Memory
Machine "1" *-- "1" CPU
Machine "1" *-- "1" ProgramLoader
CPU "1" *-- "1" RegisterFile
CPU "1" *-- "0..1" Instruction : current

' Associations
CPU "1" --> "1" Memory : reads / writes
Memory "1" --> "1" Screen : prints when\naddress = FF

' Interface realisation
CPU ..|> ExecutionContext

' Instruction hierarchy
Instruction <|-- LoadFromMemory
Instruction <|-- LoadImmediate
Instruction <|-- StoreToMemory
Instruction <|-- CopyRegister
Instruction <|-- BinaryRegisterInstruction
Instruction <|-- RotateRight
Instruction <|-- JumpIfEqual
Instruction <|-- Halt
BinaryRegisterInstruction <|-- AddTwosComplement
BinaryRegisterInstruction <|-- AddFloat
BinaryRegisterInstruction <|-- OrRegisters
BinaryRegisterInstruction <|-- AndRegisters
BinaryRegisterInstruction <|-- XorRegisters

' Dependencies
Instruction ..> ExecutionContext : uses
AddTwosComplement ..> ALU
AddFloat ..> ALU
OrRegisters ..> ALU
AndRegisters ..> ALU
XorRegisters ..> ALU
RotateRight ..> ALU
CPU ..> InstructionDecoder : decodes with
InstructionDecoder ..> Instruction : creates
InstructionDecoder ..> InvalidInstructionException : throws
ProgramLoader ..> InstructionDecoder : validates with
ProgramLoader ..> Memory : writes program to
```

```
ProgramLoader ..> ProgramLoadException : throws

' Exceptions
RuntimeError <|-- InvalidInstructionException
RuntimeError <|-- ProgramLoadException

' UI
ConsoleUI "1" --> "1" Machine
ConsoleUI "1" *-- "1" ConsoleRenderer
ConsoleRenderer ..> Machine : reads state of
@enduml
```

(Constructors of the concrete instruction classes are left out to keep the diagram readable. Each one takes the decoded operands.)

3. What each class does and why

Hardware

- **Screen:** stores everything the program has printed. Keeping it in the model means the UI only has to display it.
- **Memory:** 256 cells with read/write. It knows that address FF is the screen, so a write to FF goes to Screen::print. Because of this, the instruction 3 RFF needs no special case, since it is just a normal store.
- **RegisterFile:** the 16 registers. It checks that the index is 0 to F and throws std::out_of_range otherwise.
- **ALU:** pure calculations with no state, so the methods are static. addFloat needs two private helpers because Brookshear's float is 1 sign bit, a 3-bit exponent (excess-4) and a 4-bit mantissa.
- **ExecutionContext:** a small interface listing exactly what an instruction may do to the machine (read and write registers and memory, jump, halt). Instructions depend on this interface and not on CPU, which avoids a circular dependency and keeps the CPU's internals hidden.
- **CPU:** owns PC, IR and the registers, and runs one cycle in step(): fetch() reads 2 bytes at PC into IR and does PC += 2, decode() asks the factory for an Instruction (which validates it), and execute() calls current->execute(*this). run() repeats step() until halted. It doesn't know what any opcode means.

Instructions

- **Instruction (abstract):** one polymorphic method, execute(ctx). Each of the 12 opcodes is its own small subclass, so the CPU has no switch. Adding an opcode means adding a class (Open/Closed principle).
- **BinaryRegisterInstruction:** opcodes 5 to 9 share the shape R = S op T. This class implements execute once (Template Method) and the subclasses only supply compute(a, b), which avoids duplicated code.
- **InstructionDecoder (factory):** the only place that knows the opcode table and the validity rules. Examples: opcode 0 and D, E, F are invalid. C must be exactly C000. 4 must be 40RS. A must be AR0X. It throws InvalidInstructionException otherwise.
- **Exceptions:** they give clear, typed error reporting instead of return codes.

Loader

- **ProgramLoader:** reads the file, skips ; comments and empty lines, takes the first 4 hex digits of each line, and checks every instruction with the decoder. It also checks that the start address is even and below FF and that the program fits before FF. Only if everything is valid does it write to memory, so a bad program is rejected as a whole and memory is never half-loaded.

Facade and UI

- **Machine:** the single entry point the UI talks to. It owns the Screen, Memory, CPU and loader, so it controls their lifetimes. step() returns true while the program is still running. It exposes only const views for display.
- **ConsoleUI:** the menu and user input (parsing hex, handling exceptions, the Enter-to-continue loop for step mode, “exit or load another?”).
- **ConsoleRenderer:** all formatting (16×16 memory table, registers, PC, IR, screen). Changing the look, or replacing it with a GUI, does not touch the input logic.

Why this structure follows OOP principles

- **Single responsibility:** parsing, decoding, executing, storing and displaying are each in their own class.
- **Encapsulation:** all data members are private or protected, and the outside only sees a small public interface.
- **Polymorphism and inheritance:** the instruction hierarchy replaces conditionals.
- **Dependency inversion:** instructions depend on the ExecutionContext interface, not on the concrete CPU.
- **Low coupling:** the UI only knows Machine, and the model never prints.

Details to decide before coding

- **C++ initialization order:** in Machine, declare members in the order screen, memory, cpu, because Memory holds a reference to Screen and CPU to Memory.
- **Program end near FF:** I made the loader reject programs that would reach cell FF. Otherwise loading would print to the screen.
- **Odd jump addresses and runaway programs:** B RXY with an odd XY makes the following fetches misaligned. You can either allow it, as the original machine does, or reject it as invalid. Also consider a maximum-step guard in runAll() to stop infinite loops.
- **Screen output:** print the byte as an ASCII character. You may also want to show it in hex.

I can also draw a sequence diagram of one step() or write the file-parsing rules in more detail if that would help.

PROMPT 2

> Refactor the code toward the following architecture, adapting it to the existing project where appropriate: Contains 256 bytes, uses uint8_t addresses and values, and implements memory-mapped screen output at address 0xFF.

RESPONSE 2

I’ll check whether there’s existing code in the workspace to refactor.

Looked for files

Looked for files

There’s no code in the workspace yet (only the design from earlier), so there’s nothing to refactor. Here is the Memory part of the architecture written from scratch. It matches the PlantUML design and uses uint8_t for addresses and values, so every possible address is valid by construction and needs no range check.

Screen.h

C++

```
#pragma once
#include <cstdint>
#include <string>

class Screen {
public:
    void print(std::uint8_t value) { buffer_.push_back(static_cast<char>(value)); }
    const std::string& getContent() const { return buffer_; }
    void clear() { buffer_.clear(); }

private:
    std::string buffer_;
};
Memory.h

cpp
#pragma once
#include <array>
#include <cstddef>
#include <cstdint>
#include "Screen.h"

class Memory {
public:
    static constexpr std::size_t SIZE = 256;
    static constexpr std::uint8_t SCREEN_ADDRESS = 0xFF;

    explicit Memory(Screen& screen);

    std::uint8_t read(std::uint8_t address) const;
    void write(std::uint8_t address, std::uint8_t value);
    void clear();

private:
    std::array<std::uint8_t, SIZE> cells_{};
    Screen& screen_;
};
Memory.cpp

cpp
#include "Memory.h"

Memory::Memory(Screen& screen) : screen_(screen) {}

std::uint8_t Memory::read(std::uint8_t address) const {
    return cells_[address];   // uint8_t can only be 0..255, so it is always in range
}

void Memory::write(std::uint8_t address, std::uint8_t value) {
    cells_[address] = value;
    if (address == SCREEN_ADDRESS) {
        screen_.print(value);  // memory-mapped output: writing to FF prints
    }
}

void Memory::clear() {
    cells_.fill(0);            // does not touch the screen
}
```

Design choices

- **Memory-mapped screen:** Memory::write is the only place that knows about 0xFF. Instruction 3 RFF is therefore an ordinary StoreToMemory, with no special case in the CPU or in the instruction classes.

- **Value kept at FF:** The value is also stored in the cell, so reading FF returns the last character written. If you want FF to be write-only, skip the cells_ assignment for that address.
- **Reference to Screen:** Memory holds a Screen&, so Screen must outlive it. In Machine, declare screen before memory so the construction order is correct.
- **clear() and the screen:** clear() resets only the cells. Clear the screen separately, for example when a new program is loaded.

If you have existing code you'd like refactored toward this design, paste it here and I'll adapt it. I can also write the next classes (RegisterFile, ALU, CPU) in the same style.

PROMPT 3

> send a full code as a text like first one

RESPONSE 3

Here is the full C++17 code for the design, split by file. Nothing is saved as a file; copy each block into a file with the name shown above it.

Hex.h

```
#pragma once
#include <iomanip>
#include <sstream>
#include <string>

inline std::string toHex(unsigned value, int width) {
    std::ostringstream os;
    os << std::uppercase << std::hex << std::setw(width) << std::setfill('0') << value;
    return os.str();
}
```

Exceptions.h

```
#pragma once
#include <cstdint>
#include <stdexcept>
#include <string>
#include "Hex.h"

class InvalidInstructionException : public std::runtime_error {
public:
    explicit InvalidInstructionException(std::uint16_t raw)
        : std::runtime_error("Invalid instruction: " + toHex(raw, 4)), raw_(raw) {}
    std::uint16_t raw() const { return raw_; }

private:
    std::uint16_t raw_;
};

class ProgramLoadException : public std::runtime_error {
public:
    explicit ProgramLoadException(const std::string& message) : std::runtime_error(message) {}
};
```

Screen.h

```
#pragma once
#include <string>

class Screen {
public:
```

```
    void print(std::uint8_t value) { buffer_.push_back(static_cast<char>(value)); }
    const std::string& getContent() const { return buffer_; }
    void clear() { buffer_.clear(); }

private:
    std::string buffer_;
};
```

Memory.h

```
#pragma once
#include <array>
#include <cstddef>
#include <cstdint>
#include "Screen.h"

class Memory {
public:
    static constexpr std::size_t SIZE = 256;
    static constexpr std::uint8_t SCREEN_ADDRESS = 0xFF;

    explicit Memory(Screen& screen);

    std::uint8_t read(std::uint8_t address) const;
    void write(std::uint8_t address, std::uint8_t value);
    void clear();

private:
    std::array<std::uint8_t, SIZE> cells_{};
    Screen& screen_;
};
```

Memory.cpp

```
#include "Memory.h"

Memory::Memory(Screen& screen) : screen_(screen) {}

std::uint8_t Memory::read(std::uint8_t address) const {
    return cells_[address];  // a uint8_t is always 0..255, so always in range
}

void Memory::write(std::uint8_t address, std::uint8_t value) {
    cells_[address] = value;
    if (address == SCREEN_ADDRESS) {
        screen_.print(value);  // memory-mapped output: writing to FF prints
    }
}

void Memory::clear() {
    cells_.fill(0);  // does not touch the screen
}
```

RegisterFile.h

```
#pragma once
#include <array>
#include <cstddef>
#include <cstdint>
#include <stdexcept>

class RegisterFile {
public:
    static constexpr std::size_t COUNT = 16;

    std::uint8_t read(std::uint8_t index) const {
        checkIndex(index);
        return regs_[index];
```

```
    }
    void write(std::uint8_t index, std::uint8_t value) {
        checkIndex(index);
        regs_[index] = value;
    }
    void clear() { regs_.fill(0); }

private:
    static void checkIndex(std::uint8_t index) {
        if (index >= COUNT) throw std::out_of_range("Register index must be 0..F");
    }
    std::array<std::uint8_t, COUNT> regs_{};
};
```

ALU.h

```
#pragma once
#include <cstdint>

// Stateless utility class: only static functions.
class ALU {
public:
    ALU() = delete;

    static std::uint8_t addTwosComplement(std::uint8_t a, std::uint8_t b);
    static std::uint8_t addFloat(std::uint8_t a, std::uint8_t b);
    static std::uint8_t bitOr(std::uint8_t a, std::uint8_t b);
    static std::uint8_t bitAnd(std::uint8_t a, std::uint8_t b);
    static std::uint8_t bitXor(std::uint8_t a, std::uint8_t b);
    static std::uint8_t rotateRight(std::uint8_t value, std::uint8_t times);

private:
    // Brookshear 8-bit float: 1 sign bit, 3-bit exponent (excess 4), 4-bit mantissa (0.1xxx)
    static double floatToReal(std::uint8_t bits);
    static std::uint8_t realToFloat(double x);
};
```

ALU.cpp

```
#include "ALU.h"
#include <cmath>

std::uint8_t ALU::addTwosComplement(std::uint8_t a, std::uint8_t b) {
    return static_cast<std::uint8_t>(a + b);  // wraps modulo 256 = two's complement addition
}

std::uint8_t ALU::addFloat(std::uint8_t a, std::uint8_t b) {
    return realToFloat(floatToReal(a) + floatToReal(b));
}

std::uint8_t ALU::bitOr(std::uint8_t a, std::uint8_t b)  { return static_cast<std::uint8_t>(a | b); }
std::uint8_t ALU::bitAnd(std::uint8_t a, std::uint8_t b) { return static_cast<std::uint8_t>(a & b); }
std::uint8_t ALU::bitXor(std::uint8_t a, std::uint8_t b) { return static_cast<std::uint8_t>(a ^ b); }

std::uint8_t ALU::rotateRight(std::uint8_t value, std::uint8_t times) {
    const unsigned t = times % 8;
    if (t == 0) return value;
    return static_cast<std::uint8_t>((value >> t) | (value << (8 - t)));
}

double ALU::floatToReal(std::uint8_t bits) {
    const int sign = (bits >> 7) & 1;
    const int exponent = (bits >> 4) & 0x7;
    const int mantissa = bits & 0xF;
    if (mantissa == 0) return 0.0;
    const double value = std::ldexp(mantissa / 16.0, exponent - 4);
    return sign ? -value : value;
}
```

```
std::uint8_t ALU::realToFloat(double x) {
    if (x == 0.0) return 0;
    const int sign = (x < 0) ? 0x80 : 0;
    int exp2 = 0;
    const double fraction = std::frexp(std::fabs(x), &exp2);  // fraction in [0.5, 1)
    int mantissa = static_cast<int>(std::lround(fraction * 16));
    if (mantissa == 16) {  // rounding overflow
        mantissa = 8;
        ++exp2;
    }
    int exponent = exp2 + 4;
    if (exponent > 7) {            // too large: saturate
        exponent = 7;
        mantissa = 15;
    } else if (exponent < 0) {     // too small: underflow to zero
        return 0;
    }
    return static_cast<std::uint8_t>(sign | (exponent << 4) | mantissa);
}
```

ExecutionContext.h

```
#pragma once
#include <cstdint>

// What an instruction is allowed to do to the machine.
class ExecutionContext {
public:
    virtual ~ExecutionContext() = default;

    virtual std::uint8_t readRegister(std::uint8_t index) = 0;
    virtual void writeRegister(std::uint8_t index, std::uint8_t value) = 0;
    virtual std::uint8_t readMemory(std::uint8_t address) = 0;
    virtual void writeMemory(std::uint8_t address, std::uint8_t value) = 0;
    virtual void setProgramCounter(std::uint8_t address) = 0;
    virtual void halt() = 0;
};
```

Instruction.h

```
#pragma once
#include <cstdint>
#include <string>
#include "ExecutionContext.h"
#include "Hex.h"

class Instruction {
public:
    virtual ~Instruction() = default;

    std::uint16_t getRaw() const { return raw_; }
    std::string toHexString() const { return toHex(raw_, 4); }

    virtual void execute(ExecutionContext& ctx) = 0;

protected:
    explicit Instruction(std::uint16_t raw) : raw_(raw) {}

    // digit 0 is the opcode (most significant), digit 3 is the last one
    static std::uint8_t nibble(std::uint16_t raw, int digit) {
        return static_cast<std::uint8_t>((raw >> (12 - 4 * digit)) & 0xF);
    }
    static std::uint8_t lowByte(std::uint16_t raw) {
        return static_cast<std::uint8_t>(raw & 0xFF);
    }

    std::uint16_t raw_;
```

```
};
```

Instructions.h

```
#pragma once
#include "ALU.h"
#include "Instruction.h"

// 1 RXY : load register R from memory cell XY
class LoadFromMemory : public Instruction {
public:
    explicit LoadFromMemory(std::uint16_t raw)
        : Instruction(raw), r_(nibble(raw, 1)), address_(lowByte(raw)) {}
    void execute(ExecutionContext& ctx) override {
        ctx.writeRegister(r_, ctx.readMemory(address_));
    }

private:
    std::uint8_t r_, address_;
};

// 2 RXY : load register R with the value XY
class LoadImmediate : public Instruction {
public:
    explicit LoadImmediate(std::uint16_t raw)
        : Instruction(raw), r_(nibble(raw, 1)), value_(lowByte(raw)) {}
    void execute(ExecutionContext& ctx) override { ctx.writeRegister(r_, value_); }

private:
    std::uint8_t r_, value_;
};

// 3 RXY : store register R into memory cell XY (XY = FF prints to the screen)
class StoreToMemory : public Instruction {
public:
    explicit StoreToMemory(std::uint16_t raw)
        : Instruction(raw), r_(nibble(raw, 1)), address_(lowByte(raw)) {}
    void execute(ExecutionContext& ctx) override {
        ctx.writeMemory(address_, ctx.readRegister(r_));
    }

private:
    std::uint8_t r_, address_;
};

// 4 0RS : copy register R into register S
class CopyRegister : public Instruction {
public:
    explicit CopyRegister(std::uint16_t raw)
        : Instruction(raw), r_(nibble(raw, 2)), s_(nibble(raw, 3)) {}
    void execute(ExecutionContext& ctx) override {
        ctx.writeRegister(s_, ctx.readRegister(r_));
    }

private:
    std::uint8_t r_, s_;
};

// Opcodes 5..9 : R = S op T  (Template Method: subclasses only supply compute)
class BinaryRegisterInstruction : public Instruction {
public:
    explicit BinaryRegisterInstruction(std::uint16_t raw)
        : Instruction(raw), r(nibble(raw, 1)), s(nibble(raw, 2)), t(nibble(raw, 3)) {}
    void execute(ExecutionContext& ctx) override {
        ctx.writeRegister(r, compute(ctx.readRegister(s), ctx.readRegister(t)));
    }

protected:
```

```
    virtual std::uint8_t compute(std::uint8_t a, std::uint8_t b) = 0;

    std::uint8_t r, s, t;
};

class AddTwosComplement : public BinaryRegisterInstruction {
public:
    using BinaryRegisterInstruction::BinaryRegisterInstruction;
protected:
    std::uint8_t compute(std::uint8_t a, std::uint8_t b) override { return ALU::addTwosComplement(a, b);
}
};

class AddFloat : public BinaryRegisterInstruction {
public:
    using BinaryRegisterInstruction::BinaryRegisterInstruction;
protected:
    std::uint8_t compute(std::uint8_t a, std::uint8_t b) override { return ALU::addFloat(a, b); }
};

class OrRegisters : public BinaryRegisterInstruction {
public:
    using BinaryRegisterInstruction::BinaryRegisterInstruction;
protected:
    std::uint8_t compute(std::uint8_t a, std::uint8_t b) override { return ALU::bitOr(a, b); }
};

class AndRegisters : public BinaryRegisterInstruction {
public:
    using BinaryRegisterInstruction::BinaryRegisterInstruction;
protected:
    std::uint8_t compute(std::uint8_t a, std::uint8_t b) override { return ALU::bitAnd(a, b); }
};

class XorRegisters : public BinaryRegisterInstruction {
public:
    using BinaryRegisterInstruction::BinaryRegisterInstruction;
protected:
    std::uint8_t compute(std::uint8_t a, std::uint8_t b) override { return ALU::bitXor(a, b); }
};

// A R0X : rotate register R to the right X times
class RotateRight : public Instruction {
public:
    explicit RotateRight(std::uint16_t raw)
        : Instruction(raw), r_(nibble(raw, 1)), times_(nibble(raw, 3)) {}
    void execute(ExecutionContext& ctx) override {
        ctx.writeRegister(r_, ALU::rotateRight(ctx.readRegister(r_), times_));
    }

private:
    std::uint8_t r_, times_;
};

// B RXY : if register R == register 0, PC = XY
class JumpIfEqual : public Instruction {
public:
    explicit JumpIfEqual(std::uint16_t raw)
        : Instruction(raw), r_(nibble(raw, 1)), address_(lowByte(raw)) {}
    void execute(ExecutionContext& ctx) override {
        if (ctx.readRegister(r_) == ctx.readRegister(0)) {
            ctx.setProgramCounter(address_);
        }
    }

private:
    std::uint8_t r_, address_;
};
```

```
// C 000 : halt
class Halt : public Instruction {
public:
    explicit Halt(std::uint16_t raw) : Instruction(raw) {}
    void execute(ExecutionContext& ctx) override { ctx.halt(); }
};
```

InstructionDecoder.h

```
#pragma once
#include <cstdint>
#include <memory>
#include "Instruction.h"

class InstructionDecoder {
public:
    InstructionDecoder() = delete;

    // Throws InvalidInstructionException if the instruction is not valid.
    static std::unique_ptr<Instruction> decode(std::uint8_t high, std::uint8_t low);
    static bool isValid(std::uint8_t high, std::uint8_t low);
};
```

InstructionDecoder.cpp

```
#include "InstructionDecoder.h"
#include "Exceptions.h"
#include "Instructions.h"

bool InstructionDecoder::isValid(std::uint8_t high, std::uint8_t low) {
    switch (high >> 4) {
        case 0x1: case 0x2: case 0x3:
        case 0x5: case 0x6: case 0x7: case 0x8: case 0x9:
        case 0xB:
            return true;
        case 0x4:  // 4 0RS
            return (high & 0x0F) == 0;
        case 0xA:  // A R0X
            return (low >> 4) == 0;
        case 0xC:  // C 000
            return (high & 0x0F) == 0 && low == 0;
        default:   // 0, D, E, F
            return false;
    }
}

std::unique_ptr<Instruction> InstructionDecoder::decode(std::uint8_t high, std::uint8_t low) {
    const std::uint16_t raw = static_cast<std::uint16_t>((high << 8) | low);
    if (!isValid(high, low)) throw InvalidInstructionException(raw);

    switch (high >> 4) {
        case 0x1: return std::make_unique<LoadFromMemory>(raw);
        case 0x2: return std::make_unique<LoadImmediate>(raw);
        case 0x3: return std::make_unique<StoreToMemory>(raw);
        case 0x4: return std::make_unique<CopyRegister>(raw);
        case 0x5: return std::make_unique<AddTwosComplement>(raw);
        case 0x6: return std::make_unique<AddFloat>(raw);
        case 0x7: return std::make_unique<OrRegisters>(raw);
        case 0x8: return std::make_unique<AndRegisters>(raw);
        case 0x9: return std::make_unique<XorRegisters>(raw);
        case 0xA: return std::make_unique<RotateRight>(raw);
        case 0xB: return std::make_unique<JumpIfEqual>(raw);
        case 0xC: return std::make_unique<Halt>(raw);
        default:  throw InvalidInstructionException(raw);  // unreachable
    }
}
```

CPU.h

```
#pragma once
#include <cstddef>
#include <cstdint>
#include <memory>
#include "ExecutionContext.h"
#include "Instruction.h"
#include "Memory.h"
#include "RegisterFile.h"

class CPU : public ExecutionContext {
public:
    explicit CPU(Memory& memory);

    void restart(std::uint8_t startAddress);  // PC = start, IR = 0, not halted
    void reset();                             // clear registers and restart at 00
    void step();
    void run(std::size_t maxSteps = 100000);

    bool isHalted() const { return halted_; }
    std::uint8_t getPC() const { return pc_; }
    std::uint16_t getIR() const { return ir_; }
    const RegisterFile& getRegisters() const { return registers_; }

    // ExecutionContext
    std::uint8_t readRegister(std::uint8_t index) override;
    void writeRegister(std::uint8_t index, std::uint8_t value) override;
    std::uint8_t readMemory(std::uint8_t address) override;
    void writeMemory(std::uint8_t address, std::uint8_t value) override;
    void setProgramCounter(std::uint8_t address) override;
    void halt() override;

private:
    void fetch();
    void decode();
    void execute();

    std::uint8_t pc_ = 0;
    std::uint16_t ir_ = 0;
    bool halted_ = false;
    RegisterFile registers_;
    Memory& memory_;
    std::unique_ptr<Instruction> current_;
};
```

CPU.cpp

```
#include "CPU.h"
#include <stdexcept>
#include "Hex.h"
#include "InstructionDecoder.h"

CPU::CPU(Memory& memory) : memory_(memory) {}

void CPU::restart(std::uint8_t startAddress) {
    pc_ = startAddress;
    ir_ = 0;
    halted_ = false;
    current_.reset();
}

void CPU::reset() {
    registers_.clear();
    restart(0);
}

void CPU::step() {
    if (halted_) return;
```

```
    try {
        fetch();
        decode();
        execute();
    } catch (...) {
        halted_ = true;  // an error stops the program
        throw;
    }
}

void CPU::run(std::size_t maxSteps) {
    std::size_t steps = 0;
    while (!halted_) {
        if (steps++ >= maxSteps) {
            halted_ = true;
            throw std::runtime_error("Step limit exceeded (possible infinite loop)");
        }
        step();
    }
}

void CPU::fetch() {
    // An instruction needs two cells and must not overlap the screen cell FF.
    if (pc_ >= Memory::SCREEN_ADDRESS - 1) {
        throw std::runtime_error("Program counter out of range: " + toHex(pc_, 2));
    }
    const std::uint8_t high = memory_.read(pc_);
    const std::uint8_t low = memory_.read(static_cast<std::uint8_t>(pc_ + 1));
    ir_ = static_cast<std::uint16_t>((high << 8) | low);
    pc_ = static_cast<std::uint8_t>(pc_ + 2);
}

void CPU::decode() {
    current_ = InstructionDecoder::decode(static_cast<std::uint8_t>(ir_ >> 8),
                                          static_cast<std::uint8_t>(ir_ & 0xFF));
}

void CPU::execute() {
    current_->execute(*this);
}

std::uint8_t CPU::readRegister(std::uint8_t index) { return registers_.read(index); }
void CPU::writeRegister(std::uint8_t index, std::uint8_t value) { registers_.write(index, value); }
std::uint8_t CPU::readMemory(std::uint8_t address) { return memory_.read(address); }
void CPU::writeMemory(std::uint8_t address, std::uint8_t value) { memory_.write(address, value); }
void CPU::setProgramCounter(std::uint8_t address) { pc_ = address; }
void CPU::halt() { halted_ = true; }
```

ProgramLoader.h

```
#pragma once
#include <cstddef>
#include <cstdint>
#include <optional>
#include <string>
#include <vector>
#include "Memory.h"

class ProgramLoader {
public:
    // Loads the whole program or nothing. Returns the address just after the last byte.
    std::uint8_t load(const std::string& filename, std::uint8_t startAddress, Memory& memory);

private:
    std::vector<std::uint16_t> parseFile(const std::string& filename);
    std::optional<std::uint16_t> parseLine(const std::string& line);
    void checkStartAddress(std::uint8_t startAddress, std::size_t wordCount);
};
```

ProgramLoader.cpp

```
#include "ProgramLoader.h"
#include <cctype>
#include <fstream>
#include "Exceptions.h"
#include "InstructionDecoder.h"

std::uint8_t ProgramLoader::load(const std::string& filename, std::uint8_t startAddress,
                                 Memory& memory) {
    const std::vector<std::uint16_t> program = parseFile(filename);  // may throw
    checkStartAddress(startAddress, program.size());                 // may throw

    // Everything is valid: only now do we touch memory.
    std::uint8_t address = startAddress;
    for (const std::uint16_t word : program) {
        memory.write(address, static_cast<std::uint8_t>(word >> 8));
        memory.write(static_cast<std::uint8_t>(address + 1), static_cast<std::uint8_t>(word & 0xFF));
        address = static_cast<std::uint8_t>(address + 2);
    }
    return address;
}

std::vector<std::uint16_t> ProgramLoader::parseFile(const std::string& filename) {
    std::ifstream in(filename);
    if (!in) throw ProgramLoadException("Cannot open file: " + filename);

    std::vector<std::uint16_t> program;
    std::string line;
    int lineNo = 0;
    while (std::getline(in, line)) {
        ++lineNo;
        try {
            const std::optional<std::uint16_t> word = parseLine(line);
            if (!word) continue;  // comment or empty line
            if (!InstructionDecoder::isValid(static_cast<std::uint8_t>(*word >> 8),
                                             static_cast<std::uint8_t>(*word & 0xFF))) {
                throw ProgramLoadException("invalid instruction " + toHex(*word, 4));
            }
            program.push_back(*word);
        } catch (const ProgramLoadException& e) {
            throw ProgramLoadException("Line " + std::to_string(lineNo) + ": " + e.what());
        }
    }
    if (program.empty()) throw ProgramLoadException("The file contains no instructions");
    return program;
}

std::optional<std::uint16_t> ProgramLoader::parseLine(const std::string& line) {
    const std::size_t first = line.find_first_not_of(" \t\r\n");
    if (first == std::string::npos || line[first] == ';') return std::nullopt;
    if (line.size() - first < 4) throw ProgramLoadException("expected 4 hex digits");

    std::uint16_t value = 0;
    for (std::size_t k = 0; k < 4; ++k) {
        const char c = line[first + k];
        if (!std::isxdigit(static_cast<unsigned char>(c))) {
            throw ProgramLoadException("expected 4 hex digits");
        }
        value = static_cast<std::uint16_t>((value << 4) | std::stoi(std::string(1, c), nullptr, 16));
    }
    return value;  // anything after the 4 digits is ignored
}

void ProgramLoader::checkStartAddress(std::uint8_t startAddress, std::size_t wordCount) {
    if (startAddress % 2 != 0) {
        throw ProgramLoadException("Start address must be even");
```

```
    }
    if (startAddress >= Memory::SCREEN_ADDRESS) {
        throw ProgramLoadException("Start address must be less than FF");
    }
    // The program must end before cell FF (the screen cell).
    if (startAddress + 2 * wordCount > Memory::SCREEN_ADDRESS - 1) {
        throw ProgramLoadException("The program does not fit in memory from this address");
    }
}
```

Machine.h

```
#pragma once
#include <cstdint>
#include <string>
#include "CPU.h"
#include "Memory.h"
#include "ProgramLoader.h"
#include "Screen.h"

class Machine {
public:
    Machine();

    void loadProgram(const std::string& filename, std::uint8_t startAddress);
    void overwriteRegister(std::uint8_t index, std::uint8_t value);
    void overwriteMemory(std::uint8_t address, std::uint8_t value);
    void setPC(std::uint8_t address);
    void setPCToProgramStart();
    void resetAll();

    bool step();      // returns true while the program is still running
    void runAll();
    bool isFinished() const;

    const Memory& getMemory() const { return memory_; }
    const CPU& getCPU() const { return cpu_; }
    const Screen& getScreen() const { return screen_; }

private:
    // Declaration order matters: memory_ needs screen_, cpu_ needs memory_.
    Screen screen_;
    Memory memory_;
    CPU cpu_;
    ProgramLoader loader_;
    std::uint8_t programStart_ = 0;
};
```

Machine.cpp

```
#include "Machine.h"

Machine::Machine() : memory_(screen_), cpu_(memory_) {}

void Machine::loadProgram(const std::string& filename, std::uint8_t startAddress) {
    loader_.load(filename, startAddress, memory_);  // throws on any problem, nothing is loaded
    screen_.clear();
    programStart_ = startAddress;
    cpu_.restart(startAddress);
}

void Machine::overwriteRegister(std::uint8_t index, std::uint8_t value) {
    cpu_.writeRegister(index, value);
}

void Machine::overwriteMemory(std::uint8_t address, std::uint8_t value) {
    memory_.write(address, value);
}
```

```
void Machine::setPC(std::uint8_t address) { cpu_.restart(address); }
void Machine::setPCToProgramStart() { cpu_.restart(programStart_); }

void Machine::resetAll() {
    memory_.clear();
    screen_.clear();
    cpu_.reset();
    programStart_ = 0;
}

bool Machine::step() {
    cpu_.step();
    return !cpu_.isHalted();
}

void Machine::runAll() { cpu_.run(); }
bool Machine::isFinished() const { return cpu_.isHalted(); }
```

ConsoleRenderer.h

```
#pragma once
#include <string>
#include "CPU.h"
#include "Machine.h"
#include "Memory.h"
#include "RegisterFile.h"
#include "Screen.h"

class ConsoleRenderer {
public:
    void showState(const Machine& machine) const;
    void showMemory(const Memory& memory) const;
    void showRegisters(const RegisterFile& regs) const;
    void showPCAndIR(std::uint8_t pc, std::uint16_t ir) const;
    void showScreen(const Screen& screen) const;
    void showMessage(const std::string& text) const;
    void showError(const std::string& text) const;
};
```

ConsoleRenderer.cpp

```
#include "ConsoleRenderer.h"
#include <cctype>
#include <iostream>
#include "Hex.h"

void ConsoleRenderer::showState(const Machine& machine) const {
    std::cout << "\n==================== MACHINE STATE ====================\n";
    showMemory(machine.getMemory());
    showPCAndIR(machine.getCPU().getPC(), machine.getCPU().getIR());
    showRegisters(machine.getCPU().getRegisters());
    showScreen(machine.getScreen());
    std::cout << "=======================================================\n";
}

void ConsoleRenderer::showMemory(const Memory& memory) const {
    std::cout << "Memory (row = high digit, column = low digit):\n";
    std::cout << "     ";
    for (int c = 0; c < 16; ++c) std::cout << toHex(c, 1) << "  ";
    std::cout << "\n";
    for (int r = 0; r < 16; ++r) {
        std::cout << toHex(r, 1) << "x | ";
        for (int c = 0; c < 16; ++c) {
            const auto address = static_cast<std::uint8_t>(r * 16 + c);
            std::cout << toHex(memory.read(address), 2) << " ";
        }
        std::cout << "\n";
```

```
    }
}

void ConsoleRenderer::showPCAndIR(std::uint8_t pc, std::uint16_t ir) const {
    std::cout << "PC = " << toHex(pc, 2) << "    IR = " << toHex(ir, 4) << "\n";
}

void ConsoleRenderer::showRegisters(const RegisterFile& regs) const {
    std::cout << "Registers:\n";
    for (int i = 0; i < 16; ++i) {
        std::cout << "R" << toHex(i, 1) << "=" << toHex(regs.read(static_cast<std::uint8_t>(i)), 2)
                  << (i % 8 == 7 ? "\n" : "  ");
    }
}

void ConsoleRenderer::showScreen(const Screen& screen) const {
    std::cout << "Screen output:\n";
    for (const char c : screen.getContent()) {
        std::cout << ((std::isprint(static_cast<unsigned char>(c)) || c == '\n') ? c : '.');
    }
    std::cout << "\n(hex:";
    for (const char c : screen.getContent()) {
        std::cout << ' ' << toHex(static_cast<unsigned char>(c), 2);
    }
    std::cout << ")\n";
}

void ConsoleRenderer::showMessage(const std::string& text) const {
    std::cout << text << "\n";
}

void ConsoleRenderer::showError(const std::string& text) const {
    std::cout << "Error: " << text << "\n";
}
```

ConsoleUI.h

```
#pragma once
#include <cstdint>
#include <string>
#include "ConsoleRenderer.h"
#include "Machine.h"

class ConsoleUI {
public:
    explicit ConsoleUI(Machine& machine);
    void start();

private:
    void showMenu() const;
    void handleLoadProgram();
    void handleOverwriteRegister();
    void handleOverwriteMemory();
    void handleSetPC();
    bool handleRun();                  // returns true if the user chose to exit
    void runAllAtOnce();
    void runStepByStep();
    bool askExitOrLoadAnother() const; // returns true for "exit"

    std::uint8_t readHex(const std::string& prompt, unsigned maxValue = 0xFF) const;
    std::string readLine(const std::string& prompt) const;

    Machine& machine_;
    ConsoleRenderer renderer_;
};
```

ConsoleUI.cpp

```
#include "ConsoleUI.h"
#include <cctype>
#include <iostream>
#include "Hex.h"

namespace {
struct InputClosed {};  // thrown when stdin ends (Ctrl+D / Ctrl+Z)

bool tryParseHex(const std::string& text, unsigned& out) {
    const std::size_t b = text.find_first_not_of(" \t\r\n");
    if (b == std::string::npos) return false;
    const std::size_t e = text.find_last_not_of(" \t\r\n");
    const std::string t = text.substr(b, e - b + 1);
    if (t.size() > 2) return false;
    for (const char c : t) {
        if (!std::isxdigit(static_cast<unsigned char>(c))) return false;
    }
    out = static_cast<unsigned>(std::stoul(t, nullptr, 16));
    return true;
}
}  // namespace

ConsoleUI::ConsoleUI(Machine& machine) : machine_(machine) {}

void ConsoleUI::start() {
    try {
        bool exitRequested = false;
        while (!exitRequested) {
            showMenu();
            const std::string choice = readLine("Choice: ");
            if (choice == "1")      handleLoadProgram();
            else if (choice == "2") handleOverwriteRegister();
            else if (choice == "3") handleOverwriteMemory();
            else if (choice == "4") handleSetPC();
            else if (choice == "5") renderer_.showState(machine_);
            else if (choice == "6") exitRequested = handleRun();
            else if (choice == "7") {
                machine_.resetAll();
                renderer_.showMessage("Machine reset.");
            }
            else if (choice == "0") exitRequested = true;
            else renderer_.showError("Unknown choice.");
        }
    } catch (const InputClosed&) {
        std::cout << "\n";
    }
    renderer_.showMessage("Goodbye!");
}

void ConsoleUI::showMenu() const {
    std::cout << "\n------------- VOLE MACHINE -------------\n"
                 "1. Load a program from a file\n"
                 "2. Overwrite a register\n"
                 "3. Overwrite a memory cell\n"
                 "4. Set PC to the program start address\n"
                 "5. Show memory, PC, IR, registers and screen\n"
                 "6. Run the program\n"
                 "7. Reset the machine\n"
                 "0. Exit\n";
}

void ConsoleUI::handleLoadProgram() {
    const std::string filename = readLine("Program file: ");
    const std::uint8_t start = readHex("Start address (even hex, less than FF): ");
    try {
        machine_.loadProgram(filename, start);
        renderer_.showMessage("Program loaded. PC set to " + toHex(start, 2) + ".");
```

```
    } catch (const std::exception& e) {
        renderer_.showError(e.what());
    }
}

void ConsoleUI::handleOverwriteRegister() {
    const std::uint8_t index = readHex("Register (0-F): ", 0x0F);
    const std::uint8_t value = readHex("New value (00-FF): ");
    try {
        machine_.overwriteRegister(index, value);
        renderer_.showMessage("R" + toHex(index, 1) + " = " + toHex(value, 2));
    } catch (const std::exception& e) {
        renderer_.showError(e.what());
    }
}

void ConsoleUI::handleOverwriteMemory() {
    const std::uint8_t address = readHex("Address (00-FF): ");
    const std::uint8_t value = readHex("New value (00-FF): ");
    try {
        machine_.overwriteMemory(address, value);
        renderer_.showMessage("Cell " + toHex(address, 2) + " = " + toHex(value, 2));
    } catch (const std::exception& e) {
        renderer_.showError(e.what());
    }
}

void ConsoleUI::handleSetPC() {
    machine_.setPCToProgramStart();
    renderer_.showMessage("PC set to the program start address.");
}

bool ConsoleUI::handleRun() {
    if (machine_.isFinished()) {
        renderer_.showMessage("The program has already finished. Use option 4 to set the PC "
                              "back to the start (or load another program).");
        return false;
    }

    const std::string mode = readLine("1 = run all at once, 2 = step by step: ");
    if (mode == "1") {
        runAllAtOnce();
    } else if (mode == "2") {
        runStepByStep();
    } else {
        renderer_.showError("Unknown mode.");
        return false;
    }

    if (machine_.isFinished()) {
        if (askExitOrLoadAnother()) return true;
        handleLoadProgram();
    }
    return false;
}

void ConsoleUI::runAllAtOnce() {
    try {
        machine_.runAll();
        renderer_.showMessage("Program finished.");
    } catch (const std::exception& e) {
        renderer_.showError(e.what());
    }
    renderer_.showState(machine_);
}

void ConsoleUI::runStepByStep() {
    for (;;) {
        bool running = false;
```

```
        try {
            running = machine_.step();
        } catch (const std::exception& e) {
            renderer_.showError(e.what());
            renderer_.showState(machine_);
            return;
        }
        renderer_.showState(machine_);
        if (!running) {
            renderer_.showMessage("Program finished.");
            return;
        }
        const std::string answer = readLine("Press Enter for the next step (q = stop): ");
        if (answer == "q" || answer == "Q") return;
    }
}

bool ConsoleUI::askExitOrLoadAnother() const {
    for (;;) {
        const std::string answer = readLine("(E)xit or (L)oad another program? ");
        if (answer == "e" || answer == "E") return true;
        if (answer == "l" || answer == "L") return false;
        renderer_.showError("Please type E or L.");
    }
}

std::uint8_t ConsoleUI::readHex(const std::string& prompt, unsigned maxValue) const {
    for (;;) {
        unsigned value = 0;
        if (tryParseHex(readLine(prompt), value) && value <= maxValue) {
            return static_cast<std::uint8_t>(value);
        }
        renderer_.showError("Please enter a hex value from 00 to " + toHex(maxValue, 2) + ".");
    }
}

std::string ConsoleUI::readLine(const std::string& prompt) const {
    std::cout << prompt << std::flush;
    std::string line;
    if (!std::getline(std::cin, line)) throw InputClosed{};
    return line;
}
```

main.cpp

```
#include "ConsoleUI.h"
#include "Machine.h"

int main() {
    Machine machine;
    ConsoleUI ui(machine);
    ui.start();
    return 0;
}
```

Build and test

Shell

```
g++ -std=c++17 -Wall -Wextra -o vole main.cpp ConsoleUI.cpp ConsoleRenderer.cpp \
    Machine.cpp ProgramLoader.cpp CPU.cpp InstructionDecoder.cpp ALU.cpp Memory.cpp
./vole
```

A test program, hello.txt, that prints Hi:

hello.txt

```
; print "Hi"
2048   load R0 with 'H'
30FF   store R0 to the screen
2169   load R1 with 'i'
31FF   store R1 to the screen
C000   halt
```

In the menu, press 1, enter hello.txt and address 00, then press 6 and choose a mode. The screen section should show Hi.

Small differences from the UML

- **CPU::run(maxSteps):** It has a step limit (default 100000), so an infinite loop ends with an error instead of freezing.
- **reset() / resetAll():** I added them, with menu option 7 to use them.
- **ConsoleUI::handleRun():** It returns bool, so the “exit or load another” answer can end the program.
- **Error handling in CPU::step():** After any error, the CPU is marked as halted, so the UI knows the run is over.
- **Hex.h and Instructions.h:** Hex.h is a helper that formats numbers as hex. Instructions.h groups all 12 small instruction classes in one header.
- **Invalid-instruction errors from the loader:** They include the line number, for example Line 3: invalid instruction D123.

I can also explain any class in more detail or help you write unit tests for the ALU and the decoder.

PROMPT 4

> ok send that one can I upload it on plantuml and send it as a text

RESPONSE 4

Here is the PlantUML code, updated to match the final C++ code (reset(), resetAll(), the maxSteps limit, and handleRun() returning bool). Copy it all and paste it into the editor at https://www.plantuml.com/plantuml.

PlantUML

```
@startuml VoleMachine
skinparam classAttributeIconSize 0
skinparam shadowing false
hide empty members

' ==================== HARDWARE ====================
package "Hardware" {

  class Screen {
    - buffer : std::string
    + print(value : uint8_t) : void
    + getContent() : const std::string&
    + clear() : void
  }

  class Memory {
    + {static} SIZE : size_t = 256
    + {static} SCREEN_ADDRESS : uint8_t = 0xFF
    - cells : std::array<uint8_t, 256>
    - screen : Screen&
    + Memory(screen : Screen&)
    + read(address : uint8_t) : uint8_t
```

```
    + write(address : uint8_t, value : uint8_t) : void
    + clear() : void
  }

  class RegisterFile {
    + {static} COUNT : size_t = 16
    - regs : std::array<uint8_t, 16>
    + read(index : uint8_t) : uint8_t
    + write(index : uint8_t, value : uint8_t) : void
    + clear() : void
    - {static} checkIndex(index : uint8_t) : void
  }

  class ALU <<utility>> {
    + {static} addTwosComplement(a : uint8_t, b : uint8_t) : uint8_t
    + {static} addFloat(a : uint8_t, b : uint8_t) : uint8_t
    + {static} bitOr(a : uint8_t, b : uint8_t) : uint8_t
    + {static} bitAnd(a : uint8_t, b : uint8_t) : uint8_t
    + {static} bitXor(a : uint8_t, b : uint8_t) : uint8_t
    + {static} rotateRight(value : uint8_t, times : uint8_t) : uint8_t
    - {static} floatToReal(bits : uint8_t) : double
    - {static} realToFloat(x : double) : uint8_t
  }

  interface ExecutionContext {
    + {abstract} readRegister(index : uint8_t) : uint8_t
    + {abstract} writeRegister(index : uint8_t, value : uint8_t) : void
    + {abstract} readMemory(address : uint8_t) : uint8_t
    + {abstract} writeMemory(address : uint8_t, value : uint8_t) : void
    + {abstract} setProgramCounter(address : uint8_t) : void
    + {abstract} halt() : void
  }

  class CPU {
    - pc : uint8_t
    - ir : uint16_t
    - halted : bool
    - registers : RegisterFile
    - memory : Memory&
    - current : std::unique_ptr<Instruction>
    + CPU(memory : Memory&)
    + restart(startAddress : uint8_t) : void
    + reset() : void
    + step() : void
    + run(maxSteps : size_t = 100000) : void
    + isHalted() : bool
    + getPC() : uint8_t
    + getIR() : uint16_t
    + getRegisters() : const RegisterFile&
    + readRegister(index : uint8_t) : uint8_t
    + writeRegister(index : uint8_t, value : uint8_t) : void
    + readMemory(address : uint8_t) : uint8_t
    + writeMemory(address : uint8_t, value : uint8_t) : void
    + setProgramCounter(address : uint8_t) : void
    + halt() : void
    - fetch() : void
    - decode() : void
    - execute() : void
  }
}

' ===================== INSTRUCTIONS =====================
package "Instructions" {

  abstract class Instruction {
    # raw : uint16_t
    # Instruction(raw : uint16_t)
    + getRaw() : uint16_t
    + toHexString() : std::string
```

```
  + {abstract} execute(ctx : ExecutionContext&) : void
  # {static} nibble(raw : uint16_t, digit : int) : uint8_t
  # {static} lowByte(raw : uint16_t) : uint8_t
}

abstract class BinaryRegisterInstruction {
  # r : uint8_t
  # s : uint8_t
  # t : uint8_t
  + execute(ctx : ExecutionContext&) : void
  # {abstract} compute(a : uint8_t, b : uint8_t) : uint8_t
}

class LoadFromMemory {
  - r : uint8_t
  - address : uint8_t
  + execute(ctx : ExecutionContext&) : void
}
class LoadImmediate {
  - r : uint8_t
  - value : uint8_t
  + execute(ctx : ExecutionContext&) : void
}
class StoreToMemory {
  - r : uint8_t
  - address : uint8_t
  + execute(ctx : ExecutionContext&) : void
}
class CopyRegister {
  - r : uint8_t
  - s : uint8_t
  + execute(ctx : ExecutionContext&) : void
}
class AddTwosComplement {
  # compute(a : uint8_t, b : uint8_t) : uint8_t
}
class AddFloat {
  # compute(a : uint8_t, b : uint8_t) : uint8_t
}
class OrRegisters {
  # compute(a : uint8_t, b : uint8_t) : uint8_t
}
class AndRegisters {
  # compute(a : uint8_t, b : uint8_t) : uint8_t
}
class XorRegisters {
  # compute(a : uint8_t, b : uint8_t) : uint8_t
}
class RotateRight {
  - r : uint8_t
  - times : uint8_t
  + execute(ctx : ExecutionContext&) : void
}
class JumpIfEqual {
  - r : uint8_t
  - address : uint8_t
  + execute(ctx : ExecutionContext&) : void
}
class Halt {
  + execute(ctx : ExecutionContext&) : void
}

class InstructionDecoder <<factory>> {
  + {static} decode(high : uint8_t, low : uint8_t) : std::unique_ptr<Instruction>
  + {static} isValid(high : uint8_t, low : uint8_t) : bool
}

class "std::runtime_error" as RuntimeError <<std>>
class InvalidInstructionException {
```

```
    - raw : uint16_t
    + InvalidInstructionException(raw : uint16_t)
    + raw() : uint16_t
  }
}

' ==================== LOADER ====================
package "Loader" {
  class ProgramLoader {
    + load(filename : const std::string&, startAddress : uint8_t, memory : Memory&) : uint8_t
    - parseFile(filename : const std::string&) : std::vector<uint16_t>
    - parseLine(line : const std::string&) : std::optional<uint16_t>
    - checkStartAddress(startAddress : uint8_t, wordCount : size_t) : void
  }
  class ProgramLoadException {
    + ProgramLoadException(message : const std::string&)
  }
}

' ==================== FACADE ====================
class Machine <<facade>> {
  - screen : Screen
  - memory : Memory
  - cpu : CPU
  - loader : ProgramLoader
  - programStart : uint8_t
  + Machine()
  + loadProgram(filename : const std::string&, startAddress : uint8_t) : void
  + overwriteRegister(index : uint8_t, value : uint8_t) : void
  + overwriteMemory(address : uint8_t, value : uint8_t) : void
  + setPC(address : uint8_t) : void
  + setPCToProgramStart() : void
  + resetAll() : void
  + step() : bool
  + runAll() : void
  + isFinished() : bool
  + getMemory() : const Memory&
  + getCPU() : const CPU&
  + getScreen() : const Screen&
}

' ==================== UI ====================
package "UI" {
  class ConsoleUI {
    - machine : Machine&
    - renderer : ConsoleRenderer
    + ConsoleUI(machine : Machine&)
    + start() : void
    - showMenu() : void
    - handleLoadProgram() : void
    - handleOverwriteRegister() : void
    - handleOverwriteMemory() : void
    - handleSetPC() : void
    - handleRun() : bool
    - runAllAtOnce() : void
    - runStepByStep() : void
    - askExitOrLoadAnother() : bool
    - readHex(prompt : const std::string&, maxValue : unsigned = 0xFF) : uint8_t
    - readLine(prompt : const std::string&) : std::string
  }

  class ConsoleRenderer {
    + showState(machine : const Machine&) : void
    + showMemory(memory : const Memory&) : void
    + showRegisters(regs : const RegisterFile&) : void
    + showPCAndIR(pc : uint8_t, ir : uint16_t) : void
    + showScreen(screen : const Screen&) : void
    + showMessage(text : const std::string&) : void
    + showError(text : const std::string&) : void
```

```
  }
}

' ==================== RELATIONSHIPS ====================
' Ownership
Machine "1" *-- "1" Screen
Machine "1" *-- "1" Memory
Machine "1" *-- "1" CPU
Machine "1" *-- "1" ProgramLoader
CPU "1" *-- "1" RegisterFile
CPU "1" *-- "0..1" Instruction : current

' Associations
CPU "1" --> "1" Memory : reads / writes
Memory "1" --> "1" Screen : prints when\naddress = FF

' Interface realisation
CPU ..|> ExecutionContext

' Instruction hierarchy
Instruction <|-- LoadFromMemory
Instruction <|-- LoadImmediate
Instruction <|-- StoreToMemory
Instruction <|-- CopyRegister
Instruction <|-- BinaryRegisterInstruction
Instruction <|-- RotateRight
Instruction <|-- JumpIfEqual
Instruction <|-- Halt
BinaryRegisterInstruction <|-- AddTwosComplement
BinaryRegisterInstruction <|-- AddFloat
BinaryRegisterInstruction <|-- OrRegisters
BinaryRegisterInstruction <|-- AndRegisters
BinaryRegisterInstruction <|-- XorRegisters

' Dependencies
Instruction ..> ExecutionContext : uses
AddTwosComplement ..> ALU
AddFloat ..> ALU
OrRegisters ..> ALU
AndRegisters ..> ALU
XorRegisters ..> ALU
RotateRight ..> ALU
CPU ..> InstructionDecoder : decodes with
InstructionDecoder ..> Instruction : creates
InstructionDecoder ..> InvalidInstructionException : throws
ProgramLoader ..> InstructionDecoder : validates with
ProgramLoader ..> Memory : writes program to
ProgramLoader ..> ProgramLoadException : throws

' Exceptions
RuntimeError <|-- InvalidInstructionException
RuntimeError <|-- ProgramLoadException

' UI
ConsoleUI "1" --> "1" Machine
ConsoleUI "1" *-- "1" ConsoleRenderer
ConsoleRenderer ..> Machine : reads state of
@enduml
```

Notes

- **Tip:** In the online editor, you can download the diagram as PNG or SVG from the buttons under the image.
- **Constructors:** The constructors of the 12 concrete instruction classes are left out to keep the diagram readable. Each one takes the raw 16-bit instruction.

- **Missing helper:** Hex.h (the toHex function) is a small helper, so I didn't draw it.
- **Large diagram:** If the image is too big to read, you can split it by package. For example, keep only the "Instructions" package and its relationships in one diagram, and the rest in another.

PROMPT 5

> this design is complicated make a design that have a Basic Classes in four or five classes only

RESPONSE 5

Here is a simplified design with 5 classes. I merged the smaller parts into the bigger ones: the screen goes inside Memory, and the registers and ALU go inside CPU.

PlantUML

```
@startuml VoleMachineSimple
skinparam classAttributeIconSize 0
skinparam shadowing false
hide empty members

class Instruction {
  - raw : uint16_t
  + Instruction(high : uint8_t, low : uint8_t)
  + getOpcode() : uint8_t
  + getR() : uint8_t
  + getS() : uint8_t
  + getT() : uint8_t
  + getXY() : uint8_t
  + getRaw() : uint16_t
  + isValid() : bool
  + toString() : std::string
}

class Memory {
  + {static} SIZE : size_t = 256
  + {static} SCREEN_ADDRESS : uint8_t = 0xFF
  - cells : std::array<uint8_t, 256>
  - screenOutput : std::string
  + read(address : uint8_t) : uint8_t
  + write(address : uint8_t, value : uint8_t) : void
  + getScreenOutput() : const std::string&
  + clear() : void
}

class CPU {
  - pc : uint8_t
  - ir : Instruction
  - registers : std::array<uint8_t, 16>
  - halted : bool
  - memory : Memory&
  + CPU(memory : Memory&)
  + setPC(address : uint8_t) : void
  + getPC() : uint8_t
  + getIR() : const Instruction&
  + getRegister(index : uint8_t) : uint8_t
  + setRegister(index : uint8_t, value : uint8_t) : void
  + isHalted() : bool
  + step() : void
  + reset() : void
  - fetch() : void
  - execute(inst : const Instruction&) : void
  - addFloat(a : uint8_t, b : uint8_t) : uint8_t
  - rotateRight(value : uint8_t, times : uint8_t) : uint8_t
}
```

```
class Machine {
  - memory : Memory
  - cpu : CPU
  - programStart : uint8_t
  + Machine()
  + loadProgram(filename : const std::string&, startAddress : uint8_t) : void
  + overwriteMemory(address : uint8_t, value : uint8_t) : void
  + overwriteRegister(index : uint8_t, value : uint8_t) : void
  + setPCToProgramStart() : void
  + step() : bool
  + runAll() : void
  + isFinished() : bool
  + getMemory() : const Memory&
  + getCPU() : const CPU&
  - parseLine(line : const std::string&) : std::optional<uint16_t>
}

class UI {
  - machine : Machine&
  + UI(machine : Machine&)
  + start() : void
  - showMenu() : void
  - displayState() : void
  - displayMemory() : void
  - displayRegisters() : void
  - displayScreen() : void
  - runAllAtOnce() : void
  - runStepByStep() : void
  - askExitOrLoadAnother() : bool
}

Machine "1" *-- "1" Memory
Machine "1" *-- "1" CPU
CPU "1" --> "1" Memory : reads / writes
CPU "1" *-- "1" Instruction : IR
Machine ..> Instruction : validates\nwhile loading
UI "1" --> "1" Machine : uses

note right of Memory
  Writing to address FF
  appends the byte to screenOutput
  (memory-mapped screen).
end note

note bottom of CPU
  Errors are reported with
  std::runtime_error.
end note
@enduml
```

What each class does

Class	Responsibility
Instruction	Holds one 2-byte instruction. It splits the 4 hex digits into opcode, R, S, T and XY, and isValid() checks the opcode and its format (for example, C must be exactly C000).
Memory	Stores the 256 cells with read and write. When write is called with address FF, the value is also added to screenOutput, so printing needs no special case in the CPU.
CPU	Holds the PC, the IR and the 16 registers. step() does fetch, decode (checks isValid()), then execute, and execute() uses a switch on the opcode. The ALU operations (two's complement add, float add, OR, AND, XOR, rotate) are small private functions here.

Class	Responsibility
Machine	Owns the Memory and the CPU. It loads a program file, skips comments and empty lines, checks every instruction first, and only then writes to memory (all or nothing). It also runs step() or runAll().
UI	The menu, reading input, and displaying the 16×16 memory table, PC, IR, registers and screen. It never changes data directly; it always goes through Machine.

Why this structure

- **Easy to explain:** Each class matches a real part of the computer or a clear job, so the diagram is easy to read.
- **Encapsulation:** All attributes are private, and the UI only knows Machine.
- **Simple screen output:** Because writing to FF is handled in Memory, the instruction 3 RFF is just a normal store.
- **Instruction as its own class:** It keeps validation and decoding out of the CPU, so the CPU does not parse hex digits.

Trade-off compared with the bigger design

- CPU::execute() is a switch with 12 cases. It is shorter and simpler, but adding a new instruction means editing that function.
- The loader inside Machine is a small private helper. In the big design it was its own class.

For an assignment about basic OOP, this 5-class version is usually enough. I can also give you the C++ code for it if you want, and it will be much shorter than the first version.

PROMPT 6

> can you make it more simple??

RESPONSE 6

Here is a simpler version with 4 classes. I removed Instruction (the CPU now decodes the 4 hex digits itself) and cut the small helper methods.

PlantUML

```
@startuml VoleMachineSimple
skinparam classAttributeIconSize 0
skinparam shadowing false
hide empty members

class Memory {
  - cells : uint8_t[256]
  - screenOutput : string
  + read(address : uint8_t) : uint8_t
  + write(address : uint8_t, value : uint8_t) : void
  + getScreenOutput() : string
  + clear() : void
}

class CPU {
  - pc : uint8_t
  - ir : uint16_t
  - registers : uint8_t[16]
```

```
  - halted : bool
  + CPU(memory : Memory&)
  + step() : void
  + isValid(ir : uint16_t) : bool
  + getPC() : uint8_t
  + setPC(address : uint8_t) : void
  + getIR() : uint16_t
  + getRegister(index : uint8_t) : uint8_t
  + setRegister(index : uint8_t, value : uint8_t) : void
  + isHalted() : bool
  - fetch() : void
  - execute() : void
}

class Machine {
  - programStart : uint8_t
  + loadProgram(filename : string, startAddress : uint8_t) : bool
  + run() : void
  + runStep() : bool
  + overwriteRegister(index : uint8_t, value : uint8_t) : void
  + overwriteMemory(address : uint8_t, value : uint8_t) : void
  + setPCToStart() : void
}

class UI {
  + start() : void
  - showMenu() : void
  - displayMemory() : void
  - displayRegisters() : void
  - displayScreen() : void
}

Machine "1" *-- "1" Memory
Machine "1" *-- "1" CPU
CPU "1" --> "1" Memory : reads / writes
UI "1" --> "1" Machine : uses

note right of Memory
  Writing to address FF
  also adds the byte to
  screenOutput (the screen).
end note
@enduml
```

What each class does

Class	Responsibility
Memory	Stores the 256 cells. write to address FF also adds the value to screenOutput, so printing is just a normal store.
CPU	Holds the PC, the IR and the 16 registers. step() does fetch, then isValid() (rejects bad instructions), then execute() (a switch on the opcode, including the math and rotate operations).
Machine	Owns the Memory and the CPU. It reads the program file (skips ; comments and empty lines), checks that every instruction is valid, and only then writes it into memory. It also runs the program all at once (run) or one step at a time (runStep).
UI	Shows the menu, reads the user's choices, and displays the memory table, PC, IR, registers and screen.

Why this is simpler

- **Each class matches one clear job:** storage (Memory), processing (CPU), control and loading (Machine), and display (UI).
- **Encapsulation:** all attributes are private, and the UI only talks to Machine.
- **Fewer moving parts:** no separate Instruction, ALU or Screen classes.

Trade-offs

- CPU::execute() is one switch with 12 cases, so it will be a long function. For this assignment size, that is acceptable.
- The file parsing lives inside Machine::loadProgram, so Machine does a bit more than just coordinate. If your teacher expects strict single responsibility, you can move that into a small ProgramLoader class (that would make 5 classes).

I can write the C++ code for this version too, and it will be much shorter than the first one.